

Online Retail Database System

OnlineRetailDB

For convenience and compatibility, the code in "OnlineRetailDB_query_results_screenshots.pdf" is also provided in a separate .sql file ("OnlineRetailDB_queries.sql"), allowing it to be copied, edited, and executed directly without the formatting or encoding issues that can arise when extracting code from PDFs, particularly when lengthy code is not fully visible.

Overview

A relational database design and query project for a simulated online retail company, covering schema design, sample data, and a set of advanced SQL queries in MySQL, along with the equivalent formal expressions in Relational Algebra and Tuple Relational Calculus.

The queries move beyond basic joins and aggregates into recursive CTEs (for modeling a hierarchical product category tree), set-based "for all" logic (customers who ordered every product in a category, products supplied by every regional supplier), and window functions (ranking each customer's top spending category). A few queries required catching and correcting an initial misread of the data -- for example, an early version of the "ordered every Electronics product" query only checked the parent category, but most products actually lived in child categories like Laptops and Smartphones, which required rethinking the query with a recursive category tree instead of a flat match.

The formal language section translates two of the SQL queries into Relational Algebra (including the DIVISION operator) and Tuple Relational Calculus, working through why a plain universal quantifier isn't sufficient for the "all suppliers in a region" case without pairing it with a logical implication.

Administration

Schemas

companydb_codes*

Queries*

OnlineRetailDB*

Context Help

Snippets

MANAGEMENT

Server Status

Client Connections

Users and Privileges

Status and System Variables

Data Export

Data Import/Restore

INSTANCE

Startup / Shutdown

Server Logs

Options File

PERFORMANCE

Dashboard

Performance Reports

Performance Schema Setup

Limit to 1000 rows

⌵

★

🔍

📄

↺

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

WITH RECURSIVE ElectronicsCategoryTree AS (

-- Starting point of the recursion, just the Electronics row itself.

SELECT CategoryID FROM Categories WHERE Category_Name = 'Electronics'

UNION ALL

-- Joins the CTE back to Categories to pull in whatever is one level further down each pass,

-- stopping on its own once nothing new matches.

SELECT child_categories.CategoryID FROM Categories AS child_categories

INNER JOIN ElectronicsCategoryTree ON child_categories.Parent_Category = ElectronicsCategoryTree.CategoryID

)

-- Division by counting instead of the nested NOT EXISTS pattern. The idea: join each customer

-- to every Electronics-tree product they've actually ordered, then GROUP BY collapses that down to one row per

-- customer with a count of how many distinct products showed up. HAVING keeps only the customers whose count

-- matches the total number of Electronics-tree products that exist, since matching the full count is the only way

-- to have ordered every single one of them. Same underlying logic as Query 3, just reached through counting

-- instead of double negation.

SELECT

customers.CustomerID,

customers.First_Name,

customers.Last_Name,

customers.Email

FROM Customers AS customers

-- INNER JOIN Orders AS orders ON orders.CustomerID = customers.CustomerID AND orders.Status != 'Cancelled'

INNER JOIN Orders AS orders ON orders.CustomerID = customers.CustomerID AND orders.Status <> 'Cancelled'

INNER JOIN Order_Items AS order_items ON order_items.OrderID = orders.OrderID

INNER JOIN Products AS electronics_products ON electronics_products.ProductID = order_items.ProductID

AND electronics_products.CategoryID IN (SELECT CategoryID FROM ElectronicsCategoryTree)

GROUP BY customers.CustomerID, customers.First_Name, customers.Last_Name, customers.Email

HAVING COUNT(DISTINCT electronics_products.ProductID) = (

SELECT COUNT(*) FROM Products WHERE CategoryID IN (SELECT CategoryID FROM ElectronicsCategoryTree)

);

50%

⬆️

1:20

No object selected

Object Info

Session

Result Grid

Filter Rows:

Search

Export:

🖨️

CustomerID	First_Name	Last_Name	Email

Result Grid

Form Editor

Read Only

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Query 01

Query Completed

Administration

Schemas

companydb_codes*

Queries*

OnlineRetailDB*

Context Help

Snippets

MANAGEMENT

Server Status

Client Connections

Users and Privileges

Status and System Variables

Data Export

Data Import/Restore

INSTANCE

Startup / Shutdown

Server Logs

Options File

PERFORMANCE

Dashboard

Performance Reports

Performance Schema Setup

Limit to 1000 rows

⌵

★

🔍

📄

↺

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

WITH RECURSIVE ElectronicsCategoryTree AS (

-- Starting point of the recursion, just the Electronics row itself.

SELECT CategoryID FROM Categories WHERE Category_Name = 'Electronics'

UNION ALL

-- Joins the CTE back to Categories to pull in whatever is one level further down each pass,

-- stopping on its own once nothing new matches.

SELECT child_categories.CategoryID FROM Categories AS child_categories

INNER JOIN ElectronicsCategoryTree ON child_categories.Parent_Category = ElectronicsCategoryTree.CategoryID

)

-- Division by counting instead of the nested NOT EXISTS pattern. The idea: join each customer

-- to every Electronics-tree product they've actually ordered, then GROUP BY collapses that down to one row per

-- customer with a count of how many distinct products showed up. HAVING keeps only the customers whose count

-- matches the total number of Electronics-tree products that exist, since matching the full count is the only way

-- to have ordered every single one of them. Same underlying logic as Query 3, just reached through counting

-- instead of double negation.

SELECT

customers.CustomerID,

customers.First_Name,

customers.Last_Name,

customers.Email

FROM Customers AS customers

-- INNER JOIN Orders AS orders ON orders.CustomerID = customers.CustomerID AND orders.Status != 'Cancelled'

INNER JOIN Orders AS orders ON orders.CustomerID = customers.CustomerID AND orders.Status <> 'Cancelled'

INNER JOIN Order_Items AS order_items ON order_items.OrderID = orders.OrderID

INNER JOIN Products AS electronics_products ON electronics_products.ProductID = order_items.ProductID

AND electronics_products.CategoryID IN (SELECT CategoryID FROM ElectronicsCategoryTree)

GROUP BY customers.CustomerID, customers.First_Name, customers.Last_Name, customers.Email

HAVING COUNT(DISTINCT electronics_products.ProductID) = (

SELECT COUNT(*) FROM Products WHERE CategoryID IN (SELECT CategoryID FROM ElectronicsCategoryTree)

)

);

50%

⬆️

1:20

No object selected

Object Info

Session

Result Grid

Filter Rows:

Search

Export:

🖨️

CustomerID	First_Name	Last_Name	Email

Result Grid

Form Editor

Read Only

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Query 01

Administration

Schemas

companydb_codes*

Queries*

OnlineRetailDB*

Context Help

Snippets

MANAGEMENT

Server Status

Client Connections

Users and Privileges

Status and System Variables

Data Export

Data Import/Restore

INSTANCE

Startup / Shutdown

Server Logs

Options File

PERFORMANCE

Dashboard

Performance Reports

Performance Schema Setup

Limit to 1000 rows

⚙️ ⭐ 📄 🔍 📌 🔄

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

-- Query 2: Top 3 Categories by Total Revenue (Excluding Cancelled Orders). Identify the top 3 Category_Names that

-- have generated the highest total revenue from products sold, considering only orders with a Status of 'Shipped'

-- or 'Delivered'.

--

USE OnlineRetailDB;

SELECT

categories.Category_Name,

SUM(order_items.Quantity * order_items.Unit_Price) AS Category_Total_Revenue

FROM Categories AS categories

INNER JOIN Products AS products ON categories.CategoryID = products.CategoryID

INNER JOIN Order_Items AS order_items ON products.ProductID = order_items.ProductID

INNER JOIN Orders AS orders ON order_items.OrderID = orders.OrderID

WHERE orders.Status IN ('Shipped', 'Delivered')

-- SUM() is an aggregate function, so SQL needs GROUP BY to know which rows belong together before it can add

-- anything. Skip the GROUP BY and the SUM just collapses the entire result set into one number instead of one

-- total per category. Category_Name is technically determined by CategoryID alone, so grouping by CategoryID

-- would be enough on its own, but including both here felt like the safer move rather than leaning on that

-- relationship implicitly.

GROUP BY categories.CategoryID, categories.Category_Name

ORDER BY Category_Total_Revenue DESC

LIMIT 3;

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Object Info

Session

50%

86:16

No object selected

Result Grid

Filter Rows:

Search

Export:

Fetch rows:

	Category_Name	Category_Total_Revenue	
	Luxury Watches	3200000.00	
	Fine Jewelry	668000.00	
	Luxury Phone Cases	85000.00	

Result Grid

Form Editor

Read Only

Query Completed

Query 02

Administration

Schemas

companydb_codes*

Queries*

OnlineRetailDB*

MANAGEMENT

Server Status

Client Connections

Users and Privileges

Status and System Variables

Data Export

Data Import/Restore

INSTANCE

Startup / Shutdown

Server Logs

Options File

PERFORMANCE

Dashboard

Performance Reports

Performance Schema Setup

Limit to 1000 rows

⌵

★

👉

🔍

📄

↶

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

-- Query 4: Recursive Category Hierarchy (All Descendants). List all Category_Names that are subcategories, direct

-- or indirect, of the 'Electronics' category, including Electronics itself.

USE OnlineRetailDB;

-- Walks down the category tree starting from Electronics, through however many child categories exist beneath it,

-- Laptops, Smartphones, and any further levels if they existed. Getting the UNION ALL structured correctly took a

-- couple of tries; it's not obvious at first that the two halves are doing different jobs. The first SELECT, the

-- anchor member, just finds Electronics itself. The second SELECT, the recursive member, is the one that actually

-- repeats, joining the CTE back to Categories to pull in the next level down each time, until there are no more

-- child categories left to find. The Fibonacci example in the MySQL docs is what finally made the mechanics click:

-- https://dev.mysql.com/doc/refman/8.0/en/with.html

WITH RECURSIVE CategoryHierarchy AS (

SELECT CategoryID, Category_Name, Parent_Category, 0 AS Hierarchy_Level

FROM Categories

WHERE Category_Name = 'Electronics'

UNION ALL

SELECT

child_categories.CategoryID,

child_categories.Category_Name,

child_categories.Parent_Category,

CategoryHierarchy.Hierarchy_Level + 1 -- Without + 1, every row retrieved would stay hardcoded at 0.

FROM Categories AS child_categories

INNER JOIN CategoryHierarchy ON child_categories.Parent_Category = CategoryHierarchy.CategoryID

)

SELECT CategoryID, Category_Name, Parent_Category, Hierarchy_Level FROM CategoryHierarchy

ORDER BY Hierarchy_Level, CategoryID;

Context Help

Snippets

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Object Info

Session

No object selected

50%

⌵

1:1

Result Grid

Filter Rows:

🔍 Search

Export:

📄

	CategoryID	Category_Name	Parent_Category	Hierarchy_Level	
	1	Electronics	NULL	0	
	2	Laptops	1	1	
	3	Smartphones	1	1	
	8	Luxury Phone Cases	3	2	

Result 8

Read Only

Query 04

Administration

Schemas

companydb_codes*

Queries*

OnlineRetailDB*

Context Help

Snippets

MANAGEMENT

Server Status

Client Connections

Users and Privileges

Status and System Variables

Data Export

Data Import/Restore

INSTANCE

Startup / Shutdown

Server Logs

Options File

PERFORMANCE

Dashboard

Performance Reports

Performance Schema Setup

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

USE OnlineRetailDB;

-- The double EXISTS version of this technically works but it's checking the same relationship twice from two

-- different directions, which felt like more machinery than the question actually needs. What the condition

-- really boils down to is: does this order contain a product that shows up somewhere in the Shipped group and

-- somewhere in the Pending group at the same time. That's just an intersection of two product lists, so two small

-- CTEs plus a couple of plain joins get the same result with far less nesting.

WITH ShippedProducts AS (

SELECT DISTINCT order_items.ProductID

FROM Order_Items AS order_items

INNER JOIN Orders AS orders ON orders.OrderID = order_items.OrderID

WHERE orders.Status = 'Shipped'

),

PendingProducts AS (

SELECT DISTINCT order_items.ProductID

FROM Order_Items AS order_items

INNER JOIN Orders AS orders ON orders.OrderID = order_items.OrderID

WHERE orders.Status = 'Pending'

)

SELECT DISTINCT orders.OrderID, orders.Order_Date

FROM Orders AS orders

-- Verifies the order contains at least one item matching the Shipped products pool...

WHERE EXISTS (

SELECT 1 FROM Order_Items AS order_items

INNER JOIN ShippedProducts ON ShippedProducts.ProductID = order_items.ProductID

WHERE order_items.OrderID = orders.OrderID

)

-- ...AND at least one item matching the Pending products pool (which can be a completely different product in the same order).

AND EXISTS (

SELECT 1 FROM Order_Items AS order_items

INNER JOIN PendingProducts ON PendingProducts.ProductID = order_items.ProductID

WHERE order_items.OrderID = orders.OrderID

)

);

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Object Info

Session

No object selected

50%

4:45

Result Grid

Filter Rows:

Search

Export:

OrderID

Order_Date

Result Grid

Form Editor

Read Only

Query Completed

Query 05

Administration

Schemas

companydb_codes*

Queries*

OnlineRetailDB*

Context Help

Snippets

MANAGEMENT

Server Status

Client Connections

Users and Privileges

Status and System Variables

Data Export

Data Import/Restore

INSTANCE

Startup / Shutdown

Server Logs

Options File

PERFORMANCE

Dashboard

Performance Reports

Performance Schema Setup

Limit to 1000 rows

Icons

SQL Query Editor

SQL Query

50%

57:76

Result Grid

Filter Rows:

Search

Export:

	CustomerID	First_Name	Last_Name	Category_Name	Category_Spendi...	
	4	Robert	Chen	Luxury Watches	3200000.00	
	2	Daniel	Brooks	Luxury Watches	450000.00	
	9	Rachel	Simmons	Fine Jewelry	320000.00	
	1	Michael	Turner	Fine Jewelry	250000.00	

Result 10

Result Grid

Form Editor

Read Only

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Query 06

[illegible]